

TFTrack: A Template-Free Framework for Efficient 3D Point Cloud Tracking

Zhaofeng Hu¹, Sifan Zhou², Jiahao Nie³, Ziyu Zhao⁴, Weizi Li⁵, Ci-jyun Liang¹

Abstract—LiDAR-based 3D Single Object Tracking (3D SOT) is critical for robotic perception and navigation and aims to localize dynamic objects across frames in sparse point clouds. Existing methods, rooted in the Siamese tracking paradigm from 2D vision, rely on costly dual-input designs and excessive motion modeling guided by template priors, hindering their efficiency. Our in-depth analysis reveals: (i) the template paradigm is redundant, as the previous bounding box center encodes sufficient historical context; (ii) complex motion modeling is unnecessary, as geometric alignment provides adequate motion priors. Based on the above findings, we propose the first Template-Free Tracking framework (TFTrack). The novel framework eliminates the need for template-search pairings and operates directly on the current frame guided solely by the prior bounding box center and size. We instantiate this paradigm into three variants: TFTrack-Voxel, TFTrack-Pillar, and TFTrack-Point, to explore different 3D representations under a unified framework, ensuring flexibility across sparse and dense scenes. Extensive experiments on KITTI and nuScenes benchmarks show that TFTrack is competitive with leading template-based trackers, while reducing FLOPs by $\sim 50\%$ and running at ~ 120 FPS. By simplifying overcomplicated motion-centric designs, TFTrack establishes a new minimalist paradigm for efficient 3D point cloud tracking, paving the way for real-time and resource-efficient deployment in embedded robotic systems, such as autonomous vehicles. The **code** is available.

I. INTRODUCTION

LiDAR-based 3D Single Object Tracking (3D SOT) is pivotal for perception and navigation in mobile robots and autonomous vehicles [1]–[4]. It enables real-time localization of dynamic objects across frames using sparse point clouds. This process estimates the target’s four degree-of-freedom (DoF) pose (position (x, y, z) and yaw θ) under the common assumption that objects are upright, ground-supported, and primarily rotate around the vertical axis. For instance, a person-following robot relies on 3D SOT to reliably track and estimate the person’s 3D position relative to the robot in crowded environments, which is essential for effective following control. Similarly, during UAV landing, the UAV determines its distance to the target and poses for a secure landing. These capabilities support critical tasks like path planning [5], obstacle avoidance [6], and interactive decision-making [7].

Mainstream 3D SOT methods [8]–[14] are derived from 2D visual tracking paradigms [15]–[18] and predominantly rely on siamese-based, appearance-matching architectures. These methods employ dual-input backbones to extract features from a template and search region, followed by a correlation module to regress target poses. Despite their

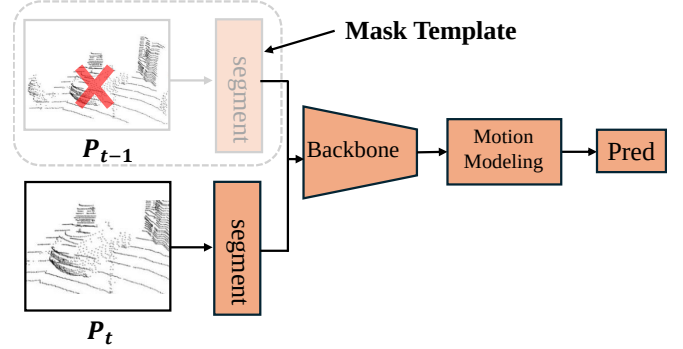


Fig. 1: Illustration of the template-based tracking architecture. The network extracts features from both the template P_{t-1} and search P_t frames through a shared backbone, and performs motion modeling for prediction. Masking the template simulates a degraded reference and reveals redundancy.

accuracy, these dual-branch siamese trackers incur substantial computational overhead through costly template-search pairings, neglecting motion cues critical for sparse or occluded point clouds. Moreover, the inherently sparse and textureless nature of LiDAR point clouds significantly weakens their representation capability under challenging conditions such as occlusions, appearance variations, and viewpoint changes, consequently limiting their robustness and generalization in dynamic environments. Recent motion-centric paradigms have emerged as robust alternatives [19]–[21]. By reformulating tracking as relative motion estimation across consecutive frames, these methods reduce reliance on appearance, improving performance in sparse environments. However, they often overcomplicate motion modeling with multi-frame inputs and uniform motion assumptions, failing to distinguish foreground dynamics from static backgrounds. Background noise complicates motion estimation, necessitating refinement modules, as in M²Track [19]. Such multi-stage pipelines and complex motion modeling hinder efficiency, especially for resource-constrained robotic platforms.

Overall, the current 3D SOT trackers, whether appearance matching or motion-centric models, face two fundamental challenges: **(1) Computational Inefficiency:** These methods process two point cloud sequences in parallel, leading to duplicated feature extraction. This doubles both FLOPs and memory footprint, making them ill-suited for the strict power and latency constraints of embedded robotic hardware. **(2) Questionable Necessity of Full Template Information:** The reliance on the template frame (previous) raises doubts about its true contribution. Specifically, it is unclear how much additional information the full template provides beyond the

¹ Stony Brook University. ² Carnegie Mellon University. ³ Hangzhou Dianzi University ⁴ Southeast University. ⁵ University of California, Riverside.

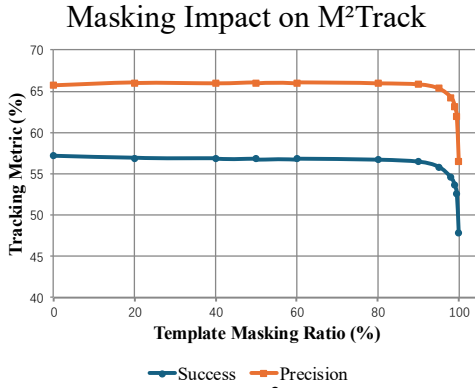


Fig. 2: Performance drop of M²Track under increasing template masking ratios on nuScenes.

TABLE I: Numerical results of M²Track under different template masking ratios on nuScenes. See Fig. 2 for the performance curve.

Mask Ratio	Success (%)	Precision (%)
0% (baseline)	57.21	65.73
20%	56.89	66.03
40%	56.86	66.01
60%	56.86	66.04
80%	56.74	66.00
100%	47.86	56.58

compact box prior, especially given the associated computational cost. To quantify how much information the template frame actually contributes, we perform a *mask-out ablation experiment*, as shown in Fig. 1 and Fig. 2. We fed a state-of-the-art motion-centric tracker [19] with the current frame while zeroing out the template input with different mask ratios. The results indicate that the average success drops by less than 1% until the mask ratio reaches 80%, as shown in Table I. This counterintuitive observation reveals: **(1) in the evaluated motion-centric setting, the prevailing Siamese dual-input paradigm introduces substantial redundancy in 3D SOT**, as the performance remains largely intact without the template input; **(2) the template point cloud is rarely indispensable for object tracking**, given the minimal performance drop when it is removed. These findings show that the template frame may not always be necessary in representative 3D SOT methods.

To this end, we present the **first template-free** framework for 3D single object tracking (TFTrack). By processing only the current point cloud with a siamese-free backbone and initializing with the prior bounding box center, TFTrack treats motion estimation as a lightweight *geometric transformation*. This design eliminates the template branch and the cross-frame matching head trims inference FLOPs by roughly 50% while maintaining competitive accuracy. We further instantiate TFTrack in three variant trackers: **TFTrack-point**, **TFTrack-voxel**, and **TFTrack-pillar**, balancing speed and performance across different 3D representations. All variants forgo the overhead of Siamese matching and achieve highly real-time speed (~ 120 FPS on a single NVIDIA 3090 GPU), making TFTrack suitable for resource-constrained robotic systems. Our main contributions are summarized as:

- **Revelation of Siamese Redundancy in 3D SOT:** Through extensive ablations on representative motion-centric Siamese 3D SOT, we reveal substantial redundancy in dense template point clouds, exposing inherent redundancy in vision-inherited Siamese networks for point cloud.
- **TFTrack: Template-Free 3D SOT Paradigm:** We propose TFTrack, the first template-free 3D tracker that reformulates 3D SOT as a lightweight geometric transformation around the prior bounding box center.
- **Unified and Efficient 3D SOT Framework:** We demonstrate the effectiveness of the unified TFTrack framework with three variants (point, voxel, and pillar) on KITTI and nuScenes datasets, achieving competitive accuracy at up to ~ 120 FPS, especially suitable for resource-constrained robotic platforms.

II. RELATED WORK

Appearance Matching for 3D SOT. Early 3D SOT methods formulate tracking as cross-frame appearance matching through Siamese architectures. They use Siamese networks for feature extraction and reveal local tracking cues based on similarity matching. While SC3D [24] pioneered template-based similarity learning, its non-end-to-end design limits candidate generation. Subsequent methods such as P2B [8] and 3D-SiamRPN [9] employ target-specific feature matching and region proposals, but template dependence introduces computational overhead. PTT [11] introduced a PointTrack-Transformer to weight critical point cloud features for enhanced appearance-based matching. In addition, PTTR [25] and STNet [26] explored different attention mechanisms to improve feature correlation. Recently, MoCUT [27] and TrackAny3D [28] introduced unified appearance-based tracking across object categories. Despite their advances, template-based methods incur high computational costs and limit real-time deployment because of their dual-input designs. Moreover, their reliance on appearance breaks down in sparse and occluded point clouds, overlooking motion cues sufficient for 3D tracking.

Motion Modeling for 3D SOT. The limitations of Siamese-based methods in handling sparse, texture-less point clouds have prompted efforts [19]–[21], [29], [30] to model inter-frame motion. M²Track [19] introduced a motion-centric framework that leverages geometric consistency across frames, reducing reliance on appearance cues. M²Track++ [20] extended this framework to a semi-supervised setting to further enhance performance. Despite improved robustness, existing methods still explicitly rely on complex multi-frame motion modeling pipelines that introduce computational redundancy. By neglecting simple geometric alignments, such as those derived from prior box centers, they limit efficiency and robustness, motivating our template-free approach.

III. METHOD

A. Overview

We define the task of LiDAR-based 3D Single Object Tracking (3D SOT) as follows: given a point-cloud sequence $\{P_t\}_{t=1}^T$ with $P_t \in \mathbb{R}^{N \times 3}$ and an initial box $B_1 =$

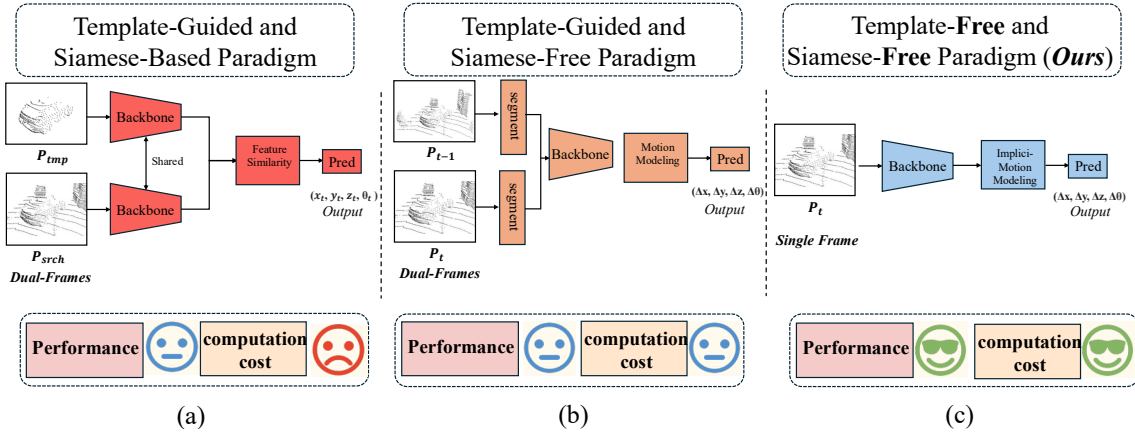


Fig. 3: Comparison of three different 3D SOT architectures. **(a)** *Template-Guided and Siamese-Based Paradigms* [8], [10], [11], [22], [23] leverage a siamese architecture for template points and search points, followed by similarity matching, yielding low efficiency. **(b)** *Template-Guided and Siamese-Free Paradigms* [19], [20] use previous and current points to model the relative motion, with sub-optimal speed and accuracy. **(c)** *Template-Free and Siamese-Free Paradigm (Ours)*: Our design removes reliance on template point clouds and Siamese backbones. It directly takes the current frame as input, implicitly encoding prior motion context in an effective one-shot pipeline, achieving high efficiency and accuracy.

$(x_1, y_1, z_1, w, h, l, \theta_1) \in \mathbb{R}^7$ in the first frame, where (x, y, z) and (w, h, l) denote the center and size, θ is the rotation angle around vertical z -axis. The goal of 3D SOT is to predict $B_t = (x_t, y_t, z_t, w, h, l, \theta_t)$ for each subsequent frame $t = 2, \dots, T$. Following [10], [11], [13], for both rigid and non-rigid objects, since the size of the target (w, h, l) remains approximately unchanged in 3D SOT across all frames, tracking reduces to regressing the 4-DoF parameters $(x_t, y_t, z_t, \theta_t) \in \mathbb{R}^4$ to represent B_t at every frame.

B. Revisit the Template-based Tracking Paradigm

Mainstream 3D trackers [11]–[13], [19], [20], [22], [31] adopt the *Siamese* paradigm with dual-input (i.e., template-search pairs), processing two point-cloud inputs via weight-shared backbones and a cross-frame prediction head. As shown in Fig 3(a)-(b), there are two dominant variants:

- **Appearance-based (template-search) tracking** [8], [11], [22] selects the point cloud from the first frame based on the ground truth box as template P^{t1} and crop points around the previous prediction B_{t-1} as search region P^s . After the siamese-based backbone, an appearance matching module F_{app} measures feature similarity to regress the 4-DoF parameters $(x_t, y_t, z_t, \theta_t)$.
- **Motion-centric (inter-frame) tracking** [19], [20] segments foreground points from dual frames ($P_{t-1}^{crop}, P_t^{crop}$) and models motion to regress the relative offset $(\Delta x, \Delta y, \Delta z, \Delta \theta)$, then applies rigid-body transformation to the previous prediction B_{t-1} to obtain $(x_t, y_t, z_t, \theta_t)$.

These process can be formulated as follows,

Template-based Appearance Matching:

$$(x_t, y_t, z_t, \theta_t) = F_{app}(P_t^{tmp}, P_t^{srch}) \quad (1)$$

Template-based Motion-Centric:

$$(x_t, y_t, z_t, \theta_t) = F_r(F_m(B_{t-1}, F_s(P_{t-1}^{crop}, P_t^{crop}))) \quad (2)$$

¹Some methods [12], [13] also adopted the previous frame as template.

where F_{app} denotes the appearance matching module, and F_s, F_m, F_r denote the segmentation module, motion inference module, and motion refine module [19], [20]. Despite their effectiveness, these methods double the input size, introduce computational redundancy in cross-frame matching, and complicate motion modeling, neglecting the sufficient geometric alignments derived from prior boxes. These limitations necessitate our *template-free*, single-frame approach.

C. Unifying Template-Free Framework for 3D SOT

As shown in Fig 3 (c), unlike existing methods, our TFTrack pioneers a template-free framework for 3D SOT, addressing the computational redundancy of template-based trackers [8], [19]. Motivated by the ablation results showing minimal performance loss without templates, we leverage the prior bounding box center as a lightweight motion prior, eliminating dual-frame inputs. We reformulate the paradigm of 3D SOT as a template-free prediction:

Template-free Motion Modeling:

$$(\Delta x, \Delta y, \Delta z, \Delta \theta) = F_{sf}(P_t^{crop}), \quad (3)$$

where P_t^{crop} is the point cloud from current frame, and F_{sf} denotes our template-free and siamese-free paradigm. This design eliminates the need for template points, cross-frame feature matching and multi-frame motion modeling, reducing both computation and memory. As shown in Fig. 4, our **TFTrack** framework unifies three variants: **TFTrack-point**, **TFTrack-voxel**, and **TFTrack-pillar**, which use point, voxel, and pillar representations for different application scenes, respectively. Each variant employs a lightweight backbone to extract geometric features, followed by a Box-Conditioned Feature Encoding module and a prediction head to regress the final position.

1) Target-Centric Transformation: Given the current frame point cloud $P_t = \{\mathbf{p}_i\}_{i=1}^N$ and the previous bounding box center $\mathbf{c}_{t-1}$, we leverage $\mathbf{c}_{t-1}$ as a location prior, defining

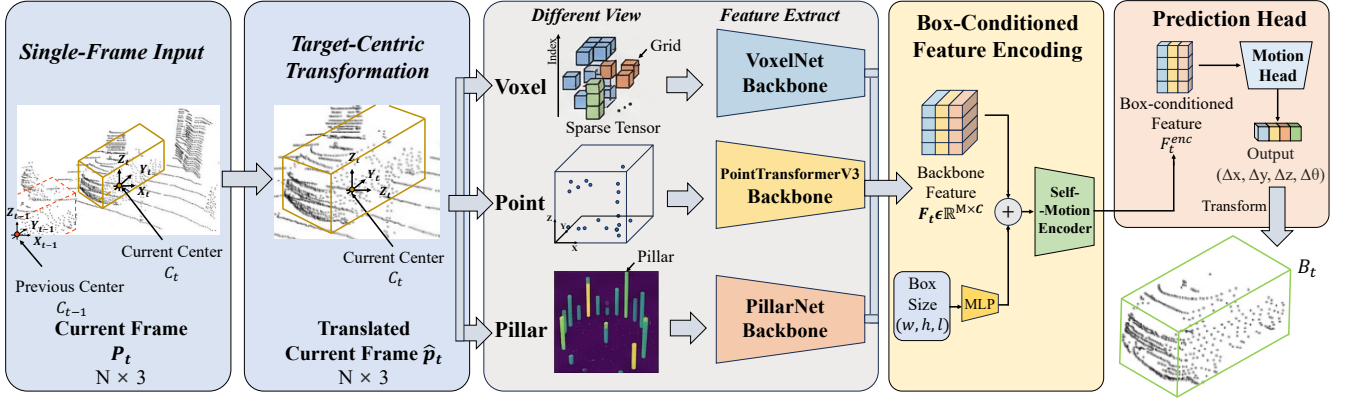


Fig. 4: Our proposed template-free tracking framework TFTrack. Given a single-frame point-cloud input, TFTrack supports point-, voxel-, and pillar-based backbones, to extract geometric features. These features are processed by Box-Conditioned Feature Encoding (BCFE) and fed into a motion head to predict relative offset, enabling efficient 3D SOT without a Siamese architecture and template input.

$\hat{P}_t = \{\mathbf{p}_i - \mathbf{c}_{t-1}\}_{i=1}^N$. This reformulates tracking as predicting a 4-DoF motion rigid transformation that aligns $\hat{P}_t$ to the target's true location in frame t . The centering operation embeds historical motion context into spatial coordinates, eliminating the need for explicit template matching while preserving geometric continuity across frames.

2) **Different Point Cloud Views:** To validate the effectiveness, geometric fidelity, and compatibility of our proposed TFTrack framework, we implement three point cloud representations: *voxel*, *point* and *pillar*. Given the input point cloud $\hat{P}_t \in \mathbb{R}^{N \times C}$, we define three representations:

- **Voxel:** The scene is partitioned into a 3D grid along X, Y, Z axes, where z-axis partitioning complicates fitting a tensor $[B, X, Y, Z, C]$ into memory. Dense grids employ a structure of size (s_x, s_y, s_z) , forming $H \times W \times D$ cells. Sparse encoding processes non-empty voxels with a separate $[M, 4]$ matrix, which stores their indices. We therefore adopt a sparse format for voxel.
- **Point:** The point cloud $\hat{P}_t$ is represented as a feature matrix of dimensions $N \times 3$, corresponding to the Cartesian coordinates (x, y, z) for each of the N points. This representation treats the input as an unordered set, retaining raw geometric information.
- **Pillar:** From a top-down view [32], [33], $\hat{P}_t$ is projected to an xy -grid, forming a 2D pseudo-image of vertical columns. Dense representations use a feature tensor $[B, X, Y, C]$, where B is the batch size, (X, Y) denote pillar counts. Sparse representations encode non-empty pillars into a matrix $[M, C]$, with an index matrix $[M, 3]$ for batch and spatial indices of non-empty pillars. We adopt a sparse format for non-empty pillars into a tensor, enabling high-speed processing.

By processing only the current frame, all variants halve FLOPs compared to Siamese or motion-centric backbone designs, while leveraging $\mathbf{c}_{t-1}$ for lightweight geometric alignment, thus redefining efficient and robust 3D tracking.

3) **Siamese-free Backbone:** We adopt a template-free architecture that encodes only the current point cloud $\hat{P}_t = \{\hat{\mathbf{p}}_i\}_{i=1}^N$. This simple shift enables the model to implicitly

capture motion context without relying on dual-frame inputs or cross-frame feature matching. A siamese-free backbone $\Phi: \mathbb{R}^{N \times 3} \rightarrow \mathbb{R}^{M \times C}$, where M denotes the number of feature tokens (e.g., voxels, points, or pillars), C denotes the backbone-dependent feature dimension, is applied to extract features in a forward pass: $\mathbf{F}_t = \Phi(\hat{P}_t) \in \mathbb{R}^{M \times C}$. Based on the input representation, the backbone can be instantiated as PointTransformerV3 [34] for point-based, VoxelNet [35] for voxel-based, and PillarNet [36] for pillar-based. We compare other backbones in Table VI. Despite architectural differences, all variants adopt the same siamese-free paradigm.

4) **Box-Conditioned Feature Encoding:** After target-centric re-centering with the previous bounding-box center, the current-frame features are represented in an aligned local coordinate system. The previous box size provides an object-aware geometric prior, allowing the network to interpret current-frame features relative to the expected target scale and shape. Given a backbone feature $\mathbf{F}_t \in \mathbb{R}^{M \times C}$, we compute a box-size embedding $\mathbf{s} = \text{MLP}(w, h, l) \in \mathbb{R}^C$ and concatenate it to each token as $\tilde{\mathbf{F}}_t = [\mathbf{F}_t \| \mathbf{1}_M \mathbf{s}^T] \in \mathbb{R}^{M \times 2C}$, where $\mathbf{1}_M$ is an all-ones column vector.

The box-conditioned feature is processed by a lightweight encoder Ψ to obtain $\mathbf{F}_t^{\text{enc}} = \Psi(\tilde{\mathbf{F}}_t)$. For point features, Ψ is implemented with point-wise feature-mixing layers; for voxel and pillar features, it uses convolutional blocks over BEV maps to capture local spatial patterns. A domain-specific readout head $R(\cdot)$ then produces the final representation $\mathbf{g}_t = R(\mathbf{F}_t^{\text{enc}})$ for offset regression. Thus, Box-Conditioned Feature Encoding (BCFE) provides scale- and shape-aware feature conditioning for target localization in the aligned current frame.

5) **Prediction Head and Training Loss:** A two-layer MLP $H: \mathbb{R}^C \rightarrow \mathbb{R}^4$ predicts the 4-DoF relative offset:

$$(\Delta x, \Delta y, \Delta z, \Delta \theta) = H(\mathbf{g}_t), \quad (x_t, y_t, z_t, \theta_t) = \text{Transform}(B_{t-1}, (\Delta x, \Delta y, \Delta z, \Delta \theta)). \quad (4)$$

At test time, the predicted bounding box is recovered via rigid body transformation, as shown in Eq. 4. Because the head consumes only a single global descriptor aggregated from the backbone features, its parameter count is identical

TABLE II: Comparisons with state-of-the-art methods on KITTI dataset. The upper and lower parts include dual-frame and single-frame trackers, respectively. *Success / Precision* are used for evaluation. † denotes reimplementaion based on official code. Mean is frame-weighted.

Paradigm	Tracker	Source	Mean (14,068)	Car (6,424)	Pedestrian (6,088)	Van (1,248)	Cyclist (308)	FPS	Device
Template-based	SC3D [24]	CVPR'19	31.2 / 48.5	41.3 / 57.9	18.2 / 37.8	40.4 / 47.0	41.5 / 70.4	2	GTx 1080Ti
	P2B [8]	CVPR'20	42.4 / 60.0	56.2 / 72.8	28.7 / 49.6	40.8 / 48.4	32.1 / 44.7	40	GTx 1080Ti
	PTT [37]	TMM'23	55.1 / 74.2	67.8 / 81.8	44.9 / 72.0	43.6 / 52.5	37.2 / 47.3	40	GTx 1080Ti
	LTTR [38]	BMVC'21	48.7 / 65.8	65.0 / 77.1	33.2 / 56.8	35.8 / 45.6	66.2 / 89.9	23	GTx 1080Ti
	BAT [10]	ICCV'21	51.2 / 72.8	60.5 / 77.7	42.1 / 70.1	52.4 / 67.0	33.7 / 45.4	57	RTx 2080
	PTTR [25]	CVPR'22	57.9 / 78.2	65.2 / 77.4	50.9 / 81.6	52.5 / 61.8	65.1 / 90.5	50	Tesla V100
	STNet [26]	ECCV'22	61.3 / 80.1	72.1 / 84.0	49.9 / 77.2	58.0 / 70.6	73.5 / 93.7	35	TITAN RTx
	GLT-T [39]	AAAI'23	60.1 / 79.3	68.2 / 82.1	52.4 / 78.8	52.6 / 62.9	68.9 / 92.1	30	GTx 1080Ti
	OSP2B [22]	IJCAI'23	60.5 / 82.3	67.5 / 82.3	53.6 / 85.1	56.3 / 66.2	65.6 / 90.5	34	GTx 1080Ti
	CXTrack [12]	CVPR'23	67.5 / 85.3	69.1 / 81.6	67.0 / 91.5	60.0 / 71.8	74.2 / 94.3	34	RTx 3090
	MBPTrack [13]	ICCV'23	70.3 / 87.9	73.4 / 84.8	68.6 / 93.9	61.3 / 72.7	76.7 / 94.3	50	RTx 3090
	M ² Track [19]	CVPR'22	62.9 / 83.4	65.5 / 80.8	61.5 / 88.2	53.8 / 70.7	73.2 / 93.5	57	Tesla V100
	M ² Track++ [20]	TPAMI'24	66.5 / 85.2	71.1 / 82.7	61.8 / 88.7	62.8 / 78.5	75.9 / 94.0	57	Tesla V100
	MoCUT [27]	ICLR'24	65.8 / 85.0	67.6 / 80.5	63.3 / 90.0	64.5 / 78.8	76.7 / 94.2	48	RTx 3070Ti
	PillarTrack [14]	arXiv'24	66.8 / 83.7	74.2 / 85.1	59.7 / 84.7	61.0 / 69.2	78.0 / 95.0	27	RTx 3080
	BEVTrack [†] [30]	IJCAI'25	71.8 / 89.2	74.9 / 86.5	69.5 / 94.3	66.0 / 77.2	77.0 / 94.7	108	RTx 3090
	TrackAny3D [28]	ICCV'25	67.1 / 85.4	73.4 / 85.2	59.6 / 85.6	70.0 / 82.8	74.7 / 94.0	28	RTx 3090
	TFTrack-Voxel	Ours	64.0 / 81.6	67.3 / 77.7	62.1 / 88.5	52.8 / 64.3	76.1 / 94.4	92	RTx 3090
	TFTrack-Point	Ours	64.4 / 83.7	70.0 / 81.1	57.7 / 86.4	66.8 / 81.9	69.5 / 92.3	121	RTx 3090
	TFTrack-Pillar	Ours	60.8 / 79.0	66.1 / 77.1	56.4 / 83.5	52.1 / 62.9	72.3 / 94.1	95	RTx 3090

across the voxel, pillar, and point variants ($< 0.1\%$ of the total), keeping the prediction-head capacity consistent across variants. To train the motion head, we adopt residual log-likelihood estimation (RLE) loss [40], which models the motion residual distribution using a flow-based density estimator:

$$L_{\text{RLE}} = -\log p_{\phi}(\mathbf{m}_t^{\text{gt}} - \mathbf{m}_t^{\text{pred}}), \quad (5)$$

where $p_{\phi}(\cdot)$ is parameterized by a conditional normalizing flow, and $\mathbf{m}_t^{\text{gt}}, \mathbf{m}_t^{\text{pred}}$ denote the ground truth and predicted motion vectors, respectively.

IV. EXPERIMENTS

A. Implementation Details

We follow the LiDAR-based 3D SOT experiments [8], [19] to evaluate TFTrack. For example, the extended spatial range is defined as $[(-4.8, 4.8), (-4.8, 4.8), (-1.5, 1.5)]$ for cars and $[(-1.92, 1.92), (-1.92, 1.92), (-1.5, 1.5)]$ for pedestrians, reflecting their different physical scales. Following backbone processing, the spatial resolution is downsampled to $16 \times 16 \times 128$. We conduct extensive experiments on the KITTI and nuScenes datasets (Table III) [41], [42]. The evaluation metrics follow standard protocols [8], [19] and report Success and Precision under the One-Pass Evaluation (OPE) setting [43], [44]. Specifically, the nuScenes dataset presents additional challenges compared to KITTI due to higher scene complexity and larger-scale annotations.

TABLE III: Comparison of KITTI and nuScenes dataset statistics.

Attribute	KITTI [41]	nuScenes [42]
Sequences/Scenes	50	1k
LiDAR Frames	19k	390k
Annotated Frames	8k	40k
Classes	8	23
Data Size	35 GB	~500 GB

B. Experimental Results

1) **Results on KITTI dataset:** We report TFTrack results on the widely adopted but limited-scale KITTI dataset. As shown in Table II, TFTrack-Voxel, TFTrack-Point and TFTrack-Pillar achieve mean Success/Precision scores of 64.0/81.6, 64.4/83.7, and 60.8/79.0, respectively. Meanwhile, TFTrack achieves strong efficiency at 92/121/95 FPS on RTX 3090, with TFTrack-Voxel further reaching 10.87 ms end-to-end latency and 185/226 MB peak allocated/reserved GPU memory. Although BEVTrack achieves higher KITTI accuracy, TFTrack offers a substantially simpler single-frame design. As summarized in Table III, KITTI is substantially smaller and less diverse than nuScenes, making it more prone to overfitting. Thus, strong KITTI performance may not fully reflect generalization to diverse large-scale environments, motivating further evaluation on nuScenes.

2) **Results on nuScenes dataset:** We evaluated TFTrack on the large-scale nuScenes dataset, known for its sparse point clouds and complex scenes. As shown in Table IV, our three variants, TFTrack-Voxel, TFTrack-Point, and TFTrack-Pillar, achieve highly competitive performance across all object categories. Notably, TFTrack-Voxel achieves 59.01/71.41 mean Success/Precision, remaining competitive with BEVTrack (59.71/71.19). TFTrack-Point achieves the best Truck and Trailer results among our variants, highlighting its strong capacity for modeling large vehicles. TFTrack-Pillar also yields solid results with a favorable balance of accuracy and efficiency. These results validate the effectiveness of our single-frame, template-free design and its ability to generalize well to dynamic and challenging tracking scenarios without dual-frame matching.

3) **Template-Free vs. Template-Based:** Table V compares our template-free *TFTrack-Voxel* against a dual-frame *Template-Based-Voxel* that processes both P_{t-1} and P_t . Despite halving the FLOPs (34.7G vs. 67.1G), the

TABLE IV: Comparisons with state-of-the-art methods on the nuScenes dataset. *Success / Precision* are used for evaluation. **Bold** denotes the best result. Mean is frame-weighted.

Paradigm	Tracker	Source	Car [64,159]	Pedestrian [33,227]	Truck [13,587]	Trailer [3,352]	Bus [2,953]	Mean [117,278]
Template-based	SC3D [24]	CVPR'19	22.31 / 21.93	11.29 / 12.65	30.67 / 27.73	35.28 / 28.12	29.35 / 24.08	20.70 / 20.20
	P2B [8]	CVPR'20	38.81 / 43.18	28.39 / 52.24	42.95 / 41.59	48.96 / 40.05	32.95 / 27.41	36.48 / 45.08
	PTT [37]	TMM'23	41.22 / 45.26	19.33 / 32.03	50.23 / 48.56	51.70 / 46.50	39.40 / 36.70	36.33 / 41.72
	BAT [10]	ICCV'21	40.73 / 43.29	28.83 / 53.32	45.34 / 42.58	52.59 / 44.89	35.44 / 28.01	38.10 / 45.71
	PTTR [25]	CVPR'22	51.89 / 58.61	29.90 / 45.09	45.30 / 44.74	45.87 / 38.36	43.14 / 37.74	44.50 / 52.07
	SMAT [45]	RAL'22	43.51 / 49.04	32.27 / 60.28	- / -	- / -	39.42 / 34.32	- / -
	STTracker [46]	RAL'23	56.11 / 69.07	37.58 / 68.36	54.29 / 60.71	36.31 / 36.07	48.13 / 55.48	49.88 / 66.61
	GLT-T [39]	AAAI'23	48.52 / 54.29	31.74 / 56.49	52.74 / 51.43	57.60 / 52.01	44.55 / 40.69	44.42 / 54.33
	FlowTrack [21]	IROS'24	60.29 / 71.07	37.60 / 67.64	- / -	55.39 / 62.70	- / -	- / -
	M ² Track [19]	CVPR'22	55.85 / 65.09	32.10 / 60.92	57.36 / 59.54	57.61 / 58.26	51.39 / 51.44	49.23 / 62.73
	MBPTrack [13]	ICCV'23	62.47 / 70.41	45.32 / 74.03	62.18 / 63.31	65.14 / 61.33	55.41 / 51.76	57.48 / 69.88
	MoCUT [27]	ICLR'24	57.32 / 66.01	33.47 / 63.12	61.75 / 64.38	60.90 / 61.84	57.39 / 56.07	51.19 / 64.63
	PillarTrack [14]	ArXiv'24	47.12 / 57.72	34.18 / 64.93	54.82 / 54.41	57.70 / 54.63	44.68 / 40.73	44.59 / 58.86
	BEVTrack [30]	IJCAI'25	64.31 / 71.14	46.28 / 76.77	66.83 / 67.04	74.54 / 71.62	61.09 / 56.68	59.71 / 71.19
	TrackAny3D [28]	ICCV'25	59.30 / 66.46	40.37 / 68.70	62.70 / 62.80	66.12 / 59.20	61.01 / 58.02	54.57 / 66.25
	TFTrack-Voxel	Ours	64.59 / 72.41	43.94 / 72.33	66.53 / 68.48	72.48 / 69.00	57.54 / 55.59	59.01 / 71.41
	TFTrack-Point	Ours	64.58 / 72.13	38.53 / 64.28	67.98 / 70.30	72.55 / 70.03	56.13 / 54.17	57.61 / 69.12
	TFTrack-Pillar	Ours	63.18 / 71.56	43.07 / 72.40	67.27 / 69.70	70.17 / 68.12	57.95 / 56.92	58.01 / 71.10

TABLE V: Comparison of **Template-Free** vs. **Template-Based** with Voxel view on nuScenes dataset (Success / Precision). Mean is frame-weighted.

Method	FLOPs (G)	Car	Ped.	Truck	Trailer	Bus	Mean
Template-Free-Voxel	34.7	64.6 / 72.4	43.9 / 72.3	66.5 / 68.5	72.5 / 69.0	57.5 / 55.6	59.0 / 71.4
Template-Based-Voxel	67.1	64.6 / 72.0	45.6 / 74.2	64.4 / 65.3	70.2 / 66.1	58.5 / 56.1	59.2 / 71.2

TABLE VI: Backbone evaluation on nuScenes dataset. **Bold** denotes the best result. Mean is frame-weighted.

Method	Version	Car	Pedestrian	Truck	Trailer	Bus	Mean
TFTrack-Voxel	VoxelNet [35]	64.6/72.4	43.9/72.3	66.5/68.5	72.5/69.0	57.5/55.6	59.0/71.4
	FocalsConv [47]	60.7/70.1	40.7/68.2	62.9/64.5	68.4/62.8	52.6/51.9	55.3/68.2
TFTrack-Point	Point-MAE [48]	64.3/72.1	38.5/64.2	66.7/69.8	71.3/69.5	55.6/53.9	57.2/69.0
	PTv3 [34]	64.6/72.1	38.5/64.3	68.0/70.3	72.6/70.0	56.1/54.2	57.6/69.1
TFTrack-Pillar	PillarNet [36]	63.2/71.6	43.1/72.4	67.3/69.7	70.2/68.1	58.0/56.9	58.0/71.1
	PointPillars [49]	59.8/68.3	40.2/69.5	63.4/66.1	66.8/65.0	54.9/53.7	54.7/67.9

template-free variant approaches the dual-frame mean Success/Precision (59.0/71.4 vs. 59.2/71.2), suggesting that the explicit template branch adds redundant computation.

4) **Backbone Analysis:** In Table VI, we benchmark three sparse backbones on nuScenes: Point-MAE vs. PTv3 [34], [48] for *TFTrack-Point*, VoxelNet vs. FocalsConv [35], [47] for *TFTrack-Voxel*, and PillarNet vs. PointPillars [36], [49] for *TFTrack-Pillar*. Upgrading to PTv3 yields a small gain (57.5/69.1 vs. 57.2/69.0), VoxelNet improves by +3.7/+3.2 over FocalsConv (59.0/71.4 vs. 55.3/68.2), and PillarNet outperforms PointPillars by +3.3/+3.2 (58.0/71.1 vs. 54.7/67.9). These results demonstrate the flexibility of TFTrack across different backbones.

5) **Ablation of Proposed Module:** In Table VII, we isolate the two core modules in *TFTrack-Voxel*. Re-centering alone raises mean Success/Precision from 50.2/62.5 to 55.1/67.6 by reducing translation variance and establishing a target-centric reference frame around B_{t-1} . BCFE alone yields a marginal gain to 50.7/62.5, as the size prior is less effective in the global coordinate system. When combined with Re-centering, BCFE further improves performance to 59.0/71.4. These results show that Re-centering provides motion-aware coordinate normalization, while BCFE supplies complementary object-scale conditioning.

6) **Template Masking:** In Table VIII, we progressively mask the template input in the dual-frame setup (0-100% for two main categories test, where masking is performed by replacing template points with zeros. Masking up to 80% causes only minor drops. This indicates that our implicit geometric priors capture the bulk of tracking cues without relying on full templates.

7) **Robustness Analysis:** We further evaluate TFTrack with tracklet-level diagnostics on nuScenes Car under challenging conditions, as shown in Table IX. We report the overall tracklet mean, late-stage performance on the last 25% of each tracklet to measure drift accumulation, sparse-initial cases to evaluate weak geometric observations, and fast-start cases to examine large initial motion. TFTrack achieves the best Precision in all settings and the best Success on the overall, late-stage, and fast-start subsets. These results show that the proposed previous-box prior remains robust under common short-term tracking challenges while preserving the template-free design.

8) **Visualization:** Fig. 5 illustrates tracking results on the nuScenes dataset. We compare three Template-free variants, TFTrack-Voxel (red), TFTrack-Point (blue), and TFTrack-Pillar (yellow), against the motion-centric M²Track (magenta) and ground truth (green). Across diverse object

TABLE VII: Ablation study of re-centering and Box-Conditioned Feature Encoding in **TFTrack-Voxel** on nuScenes dataset [42]. "Re-center" denotes the coordinate transformation of the current frame using the previous box center. "BCFE" denotes Box-Conditioned Feature Encoding. Mean is frame-weighted.

Method	#	Re-center	BCFE	Car	Pedestrian	Truck	Trailer	Bus	Mean
TF	①	✓	✓	64.6/72.4	43.9/72.3	66.5/68.5	72.5/69.0	57.5/55.6	59.0/71.4
	②	✓		61.1/69.2	40.2/68.7	61.1/62.5	66.9/61.8	52.4/50.3	55.1/67.6
	③		✓	55.8/62.8	36.5/64.8	58.2/59.0	63.5/60.1	49.1/47.2	50.7/62.5
	④			55.1/62.7	35.9/65.4	58.8/58.3	64.1/59.7	48.7/46.9	50.2/62.5

TABLE VIII: Effect of template masking on **Template-Based-Voxel (TB)** under different masking ratios on nuScenes (Success / Precision).

Method	Mask Ratio	Car	Pedestrian
TB	0% (Full Template)	64.6 / 72.0	45.6 / 74.2
	20% Masked	64.5 / 71.8	44.8 / 73.7
	40% Masked	64.2 / 71.1	44.2 / 73.6
	60% Masked	63.8 / 71.2	43.2 / 73.2
	80% Masked	62.7 / 70.3	41.3 / 70.2
	100% Masked	61.9 / 69.8	39.8 / 68.1

TABLE IX: Tracklet-level robustness under challenging conditions on nuScenes Car (Success / Precision). "Late 25%" denotes the last quarter of each tracklet; "Sp. init." denotes fewer than 10 target points in the first frame; "Fast start" denotes initial GT speed larger than 10 m/s.

Method	Mean	Late 25%	Sp. init.	Fast start
M ² Track	60.81 / 68.73	45.38 / 53.77	51.23 / 60.46	14.38 / 13.05
MBPTrack	65.72 / 73.06	51.56 / 60.14	57.98 / 66.09	12.59 / 11.50
BEVTrack	67.72 / 73.77	52.83 / 60.98	62.37 / 69.16	9.75 / 7.79
TFTrack	67.79 / 74.46	54.27 / 63.16	61.76 / 69.45	15.19 / 14.17

classes (Car, Pedestrian, Truck, Trailer, Bus), TFTrack consistently aligns closely with ground truth, showing robust tracking performance without a template point-cloud input.

V. CONCLUSION

This paper presents **TFTrack**, the first template-free and single-frame method for 3D SOT. Departing from motion-centric dual-frame paradigms, TFTrack leverages a simple yet powerful insight: the previous bounding box center and box size provide sufficient geometric prior for motion estimation from the current frame alone. By reformulating 3D SOT as a single-frame transformation problem, TFTrack eliminates redundant template encoding and cross-frame feature correlation, reducing computation and latency. We instantiate the framework with three views, point, pillar, and voxel, to explore trade-offs between accuracy and efficiency. Extensive experiments show that TFTrack achieves competitive accuracy with high inference efficiency, making it well-suited for onboard deployment in embedded systems.

Limitations: TFTrack may degrade under extremely sparse LiDAR observations, severe occlusion, or adverse weather. Following standard 3D SOT benchmarks, the current implementation predicts 4-DoF rigid motion with constant object size, which limits handling of strong deformation. Future work will explore multimodal sensing, lightweight recovery mechanisms, and richer motion representations.

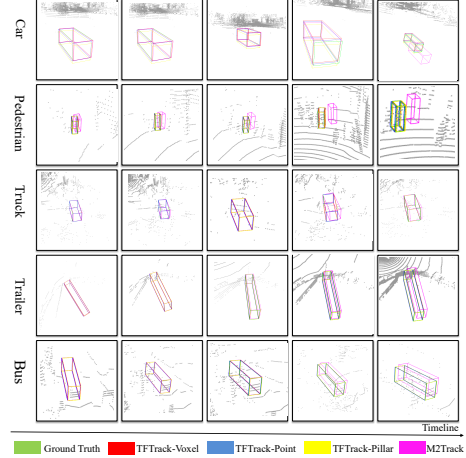


Fig. 5: Visualization of tracking results on nuScenes dataset.

REFERENCES

- [1] Y. Hu, S. Fang, W. Xie, and S. Chen, "Aerial monocular 3d object detection," *IEEE Robotics and Automation Letters*, vol. 8, no. 4, pp. 1959–1966, 2023.
- [2] Z. Li, Y. Lin, Y. Cui, S. Li, and Z. Fang, "Motion-to-matching: A mixed paradigm for 3d single object tracking," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1468–1475, 2024.
- [3] Y. Cui, Z. Fang, and S. Zhou, "Point siamese network for person tracking using 3d point clouds," *Sensors*, vol. 20, no. 1, 2020.
- [4] Z. Hu, S. Zhou, Z. Yuan, D. Yang, S. Zhao, and C.-j. Liang, "Mvctrack: Boosting 3d point cloud tracking via multimodal-guided virtual cues," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 3745–3751.
- [5] Y. Han, J. Banfi, and M. Campbell, "Planning paths through unknown space by imagining what lies therein," in *Conference on Robot Learning*. PMLR, 2021, pp. 905–914.
- [6] D. Snyder, M. Booker, N. Simon, W. Xia, D. Suo, E. Hazan, and A. Majumdar, "Online learning for obstacle avoidance," in *Conference on Robot Learning*. PMLR, 2023, pp. 2926–2954.
- [7] V. V. Unhelkar, S. Li, and J. A. Shah, "Semi-supervised learning of decision-making models for human-robot collaboration," in *conference on Robot Learning*. PMLR, 2020, pp. 192–203.
- [8] H. Qi, C. Feng, Z. Cao, F. Zhao, and Y. Xiao, "P2b: Point-to-box network for 3d object tracking in point clouds," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 6329–6338.
- [9] Z. Fang, S. Zhou, Y. Cui, and S. Scherer, "3d-siamrpn: An end-to-end learning method for real-time 3d single object tracking using raw point cloud," *IEEE Sensors Journal*, vol. 21, no. 4, pp. 4995–5011, 2021.
- [10] C. Zheng, X. Yan, J. Gao, W. Zhao, W. Zhang, Z. Li, and S. Cui, "Box-aware feature enhancement for single object tracking on point clouds," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 13 199–13 208.
- [11] J. Shan, S. Zhou, Z. Fang, and Y. Cui, "Ptt: Point-track-transformer module for 3d single object tracking in point clouds," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 1310–1316.
- [12] T.-X. Xu, Y.-C. Guo, Y.-K. Lai, and S.-H. Zhang, "Cxtrack: Improving 3d point cloud tracking with contextual information," in *Proceedings*

of the *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 1084–1093.

- [13] T. X. Xu, Y. C. Guo, Y. K. Lai, and S. H. Zhang, “Mbptrack: Improving 3d point cloud tracking with memory networks and box priors,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 9911–9920.
- [14] W. Xu, S. Zhou, and Z. Yuan, “Pillartrack: Redesigning pillar-based transformer network for single object tracking on point clouds,” *arXiv preprint arXiv:2404.07495*, 2024.
- [15] C. Ma, X. Yang, C. Zhang, and M.-H. Yang, “Long-term correlation tracking,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 5388–5396.
- [16] M. Danelljan, G. Bhat, F. Shahbaz Khan, and M. Felsberg, “Eco: Efficient convolution operators for tracking,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 6638–6646.
- [17] B. Li, J. Yan, W. Wu, Z. Zhu, and X. Hu, “High performance visual tracking with siamese region proposal network,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8971–8980.
- [18] L. Bertinetto, J. Valmadre, J. F. Henriques, A. Vedaldi, and P. H. Torr, “Fully-convolutional siamese networks for object tracking,” in *Computer Vision–ECCV 2016 Workshops: Amsterdam, The Netherlands, October 8–10 and 15–16, 2016, Proceedings, Part II 14*. Springer, 2016, pp. 850–865.
- [19] C. Zheng, X. Yan, H. Zhang, B. Wang, S. Cheng, S. Cui, and Z. Li, “Beyond 3d siamese tracking: A motion-centric paradigm for 3d single object tracking in point clouds,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 8111–8120.
- [20] —, “An effective motion-centric paradigm for 3d single object tracking in point clouds,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 1, pp. 43–60, 2024.
- [21] S. Li, Y. Cui, Z. Li, and Z. Fang, “Flowtrack: Point-level flow network for 3d single object tracking,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 427–434.
- [22] J. Nie, Z. He, Y. Yang, Z. Bao, M. Gao, and J. Zhang, “Osp2b: One-stage point-to-box network for 3d siamese tracking,” in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, 2023, pp. 1285–1293.
- [23] L. Hui, L. Wang, M. Cheng, J. Xie, and J. Yang, “3d siamese voxel-to-bev tracker for sparse point clouds,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 28 714–28 727, 2021.
- [24] S. Giancola, J. Zarzar, and B. Ghanem, “Leveraging shape completion for 3d siamese tracking,” in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2019, pp. 1359–1368.
- [25] C. Zhou, Z. Luo, Y. Luo, T. Liu, L. Pan, Z. Cai, H. Zhao, and S. Lu, “Ptr: Relational 3d point cloud object tracking with transformer,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 8531–8540.
- [26] L. Hui, L. Wang, L. Tang, K. Lan, J. Xie, and J. Yang, “3d siamese transformer network for single object tracking on point clouds,” in *Computer Vision–ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part II*. Springer, 2022, pp. 293–310.
- [27] J. Nie, Z. He, X. Lv, X. Zhou, D.-K. Chae, and F. Xie, “Towards category unification of 3d single object tracking on point clouds,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [28] M. Wang, H. Wang, Y. Li, X. Kong, J. Du, G. Shen, and F. Xia, “Trackany3d: Transferring pretrained 3d models for category-unified 3d point cloud tracking,” in *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2025, pp. 28 249–28 259.
- [29] Y. Xia, Q. Wu, W. Li, A. B. Chan, and U. Stilla, “A lightweight and detector-free 3d single object tracker on point clouds,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 5, pp. 5543–5554, 2023.
- [30] Y. Yang, Y. Deng, M. Pan, Z.-J. Zha, and J. Zhang, “Bevtrack: A simple and strong baseline for 3d single object tracking in bird’s-eye view,” in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, J. Kwok, Ed. International Joint Conferences on Artificial Intelligence Organization, 8 2025, pp. 2242–2250, main Track.
- [31] M. Wang, T. Ma, X. Zuo, J. Lv, and Y. Liu, “Correlation pyramid network for 3d single object tracking,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 3216–3225.
- [32] S. Zhou, X. Zhang, X. Chu, B. Zhang, Z. Zhao, and X. Lu, “Fastpillars: A deployment-friendly pillar-based 3d detector,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 36, no. 4, pp. 4999–5010, 2026.
- [33] S. Zhou, Z. Yuan, D. Yang, X. Hu, J. Qian, and Z. Zhao, “Pillarhist: A quantization-aware pillar feature encoder based on height-aware histogram,” in *Proceedings of the computer vision and pattern recognition conference*, 2025, pp. 27 336–27 345.
- [34] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, and H. Zhao, “Point transformer v3: Simpler, faster, stronger,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2024, pp. 4840–4851.
- [35] Y. Zhou and O. Tuzel, “Voxelnet: End-to-end learning for point cloud based 3d object detection,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4490–4499.
- [36] G. Shi, R. Li, and C. Ma, “Pillarnet: Real-time and high-performance pillar-based 3d object detection,” in *European Conference on Computer Vision*. Springer, 2022, pp. 35–52.
- [37] J. Shan, S. Zhou, Y. Cui, and Z. Fang, “Real-time 3d single object tracking with transformer,” *IEEE Transactions on Multimedia*, vol. 25, pp. 2339–2353, 2023.
- [38] Y. Cui, Z. Fang, J. Shan, Z. Gu, and S. Zhou, “3d object tracking with transformer,” in *32nd British Machine Vision Conference 2021, BMVC 2021, Online, November 22–25, 2021*. BMVA Press, 2021, p. 317.
- [39] J. Nie, Z. He, Y. Yang, M. Gao, and J. Zhang, “Glt-t: Global-local transformer voting for 3d single object tracking in point clouds,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023, pp. 1957–1965.
- [40] J. Li, S. Bian, A. Zeng, C. Wang, B. Pang, W. Liu, and C. Lu, “Human pose regression with residual log-likelihood estimation,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 11 025–11 034.
- [41] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? the kitti vision benchmark suite,” in *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 2012, pp. 3354–3361.
- [42] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, “nuscenes: A multimodal dataset for autonomous driving,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 11 621–11 631.
- [43] Y. Wu, J. Lim, and M.-H. Yang, “Online object tracking: A benchmark,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2013, pp. 2411–2418.
- [44] M. Kristan, J. Matas, A. Leonardis, T. Vojř, R. Pflugfelder, G. Fernandez, G. Nebel, F. Porikli, and L. Čehovin, “A novel performance evaluation methodology for single-target trackers,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 38, no. 11, pp. 2137–2155, 2016.
- [45] Y. Cui, J. Shan, Z. Gu, Z. Li, and Z. Fang, “Exploiting more information in sparse point cloud for 3d single object tracking,” *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11 926–11 933, 2022.
- [46] Y. Cui, Z. Li, and Z. Fang, “Stracker: Spatio-temporal tracker for 3d single object tracking,” *IEEE Robotics and Automation Letters*, vol. 8, no. 8, pp. 4967–4974, 2023.
- [47] Y. Chen, Y. Li, X. Zhang, J. Sun, and J. Jia, “Focal sparse convolutional networks for 3d object detection,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 5428–5437.
- [48] Y. Pang, W. Wang, F. E. Tay, W. Liu, Y. Tian, and L. Yuan, “Masked autoencoders for point cloud self-supervised learning,” in *Computer Vision–ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part II*. Springer, 2022, pp. 604–621.
- [49] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “Pointpillars: Fast encoders for object detection from point clouds,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 12 697–12 705.